DEV ∞ OPS

COLLABORATE & DEPLOY

CLOUD-NATIVE MOVIE TICKET BOOKING SYSTEM WITH AUTOMATED CI/CD PIPELINE USING DOCKER, JENKINS, AND AWS

A DevOps Project

Rishyab Huded 02FE23BCS077

Amey Bidikar 02FE22BCS014

Sagar Suntakar 02FE24BCS405

Kiran BPatil 02FE23BCS129

Project Advisor:
Priyanka Gavade
Department of Computer Science

01 BACKGROUND

CineBook (BookMyScreen) is a high-performance full-stack web application designed for theater ticket reservations. Built on the **MERN stack** (React.js, Node.js, Express, MongoDB), the application features concurrent seat locking, user authentication, and responsive seat layouts.

To operationalize the application, DevOps practices were introduced to automate the manual software release cycle. An automated pipeline handles builds, environment variable configuration, dockerization, Ansible deployments, and server health monitoring.

02 PROBLEM STATEMENT

Manual deployment of multi-tier web applications leads to:

- **Environment Drift**: Mismatched Node.js/package versions between local development and AWS EC2 servers.
- **Security Exposures**: Manually writing .env files or exposing production database credentials inside repositories.
- **Release Downtime**: Manual server updates cause application service disruptions.
- **Monitoring Deficits**: No instant alerts or metrics for service failures.

03 OBJECTIVES

- **Automate Pipeline**: Establish a Jenkins CI/CD pipeline triggered automatically via Git hooks upon code changes.
- **Enforce Containerization**: Package Frontend (React/Nginx), Backend (Node), and MongoDB into distinct Docker containers.
- **Define Configuration-as-Code**: Use Ansible to automate server provisioning, dependency installation, and swap file settings.
- **Implement Zero-Downtime Deployment**: Configure Nginx reverse proxy routes with automatic health verification loops.
- **Establish Monitoring & Rollback**: Deploy health check scripts to confirm backend responsiveness and support rapid rollbacks.

04 REQUIREMENT SPECIFICATIONS

HARDWARE CONFIGURATION:

- **AWS EC2 Server**: c7i-flex.large Instance (2 vCPUs with 1 core each, 4GB RAM).
- **Storage & Network**: 20GB General Purpose SSD (gp3); 1Gbps Bandwidth.
- **Virtual Swap File**: 2GB Configured Swap for npm build compilation stability.

SOFTWARE DEPLOYMENT MATRIX:

- **Operating System**: Ubuntu Server 22.04 LTS (HVM).
- **CI/CD & Provisioning**: Jenkins LTS v2.440+, Ansible Core v2.12+.
- **Container System**: Docker Engine v24.0.7+, Docker Compose v2.22.0+.
- **Server Runtimes**: Node.js v20.x, Nginx stable-alpine, MongoDB v7.0.

05 FUNCTIONAL REQUIREMENTS

- **Auto-Checkout & Build**: Detect GitHub commits via Jenkins polling (every 2 mins), checkout, and run test suites.
- **Secrets Configuration**: Inject environment variables (.env files) on target servers without storing them in Git.
- **Container Orchestration**: Docker Compose manages private networking between containers, avoiding host port clutter.
- **Reverse Proxy Proxying**: Nginx routes traffic securely on Port 80, routing `/api/v1` calls to the Node container.
- **Deployment Health Verification**: Script polls API endpoints (12 attempts) before declaring build success.

06 SYSTEM ARCHITECTURE

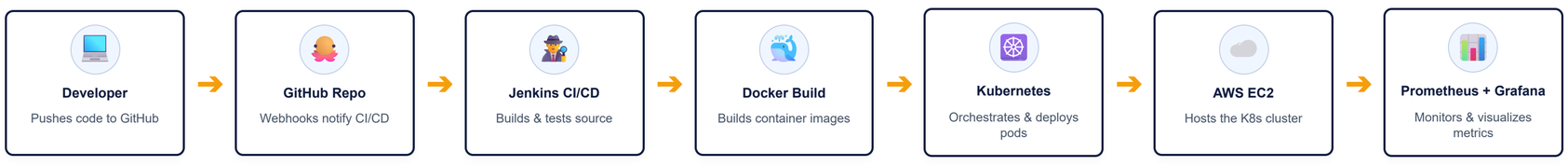


Figure 6.1: Automated CI/CD Lifecycle showing continuous integration, provisioning orchestration, container orchestration, and production monitoring.

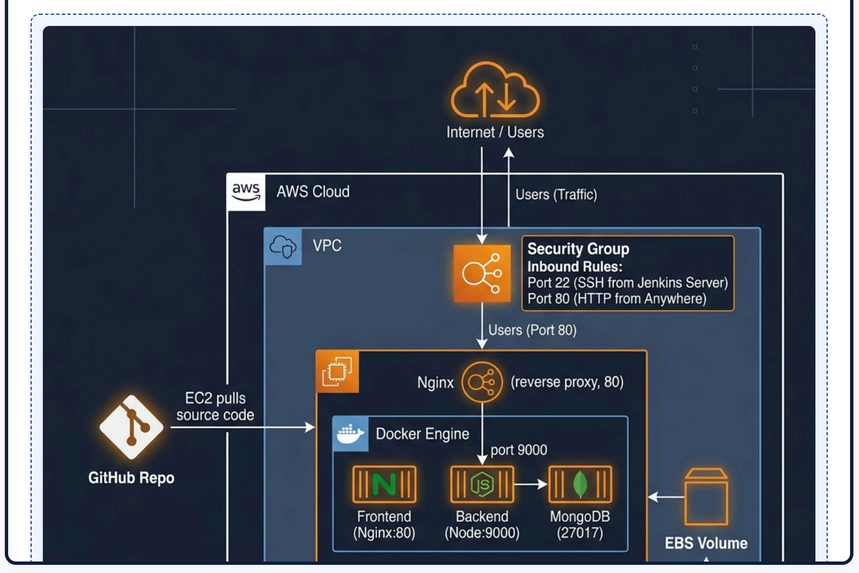
07 DEVOPS WORKFLOW

1. **Code Commits & Webhook Trigger**
   Developers collaborate on feature branches. On merging into main branch, Jenkins triggers build stage.
2. **Multi-tier Dependency Resolution**
   Jenkins server executes 'NPM Install' in subfolders to compile TypeScript and create optimized React assets.
3. **Ansible Playbook Invocation**
   Jenkins safely logs into AWS EC2 over SSH using credentials and triggers localized Ansible playbook `deploy.yml`.
4. **Host Configuration & Swap Tuning**
   Ansible checks if Docker is installed. It configures a 2GB virtual swap file to prevent memory exhaustion during builds.
5. **Docker Compose Containerization**
   Playbook runs `docker compose up -d --build`. A brand new frontend container with custom Nginx reverse proxy is built.
6. **Kubernetes Pod Deployment**
   Docker images are deployed as Kubernetes pods with replicas, service discovery, and rolling update strategies.

08 TOOLS & TECHNOLOGIES

- **GitHub / Git**
  Used for source code management, code collaboration, pull requests, and semantic version releases.
- **Jenkins Pipeline**
  Orchestrates the build cycle. Triggers tasks in sequence (Checkout, Build, Deploy, Health Check).
- **Ansible Playbook**
  Automates system installation packages, SSH authentication, swap space, and Docker Compose starts.
- **Docker Engine**
  Isolates microservices into containers. Ensures runtime configurations remain consistent everywhere.
- **Docker Compose**
  Manages multi-container structures. Connects web UI, Express API, and database in local networks.
- **Nginx Proxy**
  Serves React static build files efficiently and maps `/api/v1` routes to the backend container to bypass CORS.
- **AWS EC2 Cloud**
  Elastic Cloud Compute hosting virtual instances of Ubuntu Server to serve production loads.
- **Kubernetes**
  Container orchestration for automated pod deployment, scaling, rolling updates, and self-healing.
- **Prometheus**
  Metrics collection and time-series monitoring with alerting rules for service health and performance.
- **Grafana**
  Visualization and dashboards for real-time monitoring of application metrics and infrastructure health.

09 UML / ARCHITECTURE DIAGRAM



10 DEPLOYMENT STAGE VALIDATION

ID	Deployment Stage	Verification Method	Result
TS-01	SCM Checkout	Jenkins checkout Git revision matching main commit HEAD.	SUCCESS
TS-02	Dependency Resolve	NPM install executes with exit code 0 on Frontend and Backend.	SUCCESS
TS-03	Production Build	TypeScript builds backend, Vite bundle compiles frontend static assets.	SUCCESS
TS-04	SSH Key Exchange	Jenkins establishes secure credentials connection to EC2 remote host.	SUCCESS
TS-05	Ansible Playbook	Executes locally on host to pull updates and setup configuration.	SUCCESS
TS-06	Containers Startup	Verify container states: Mongo, Backend, and Nginx are up.	SUCCESS
TS-07	Health Check Loop	cURL verification to `/api/v1/movies` returns HTTP Code 200.	SUCCESS

11 PIPELINE EXECUTION RESULTS & AUTOMATION OUTPUT

1. Jenkins Pipeline

```
jenkins_stage_view

[Stage View] CineBook-Pipeline
-----------------------------------
✓ Checkout            [  8s ]
✓ Backend Install     [ 22s ]
✓ Frontend Install    [ 35s ]
✓ Backend Build       [ 12s ]
✓ Frontend Build      [ 28s ]
✓ Ansible Deploy      [ 45s ]
✓ Verify Deployment   [ 10s ]
-----------------------------------
SUCCESS (Total Duration: 2m 40s)
```

2. Ansible Task Execution

```
ansible-playbook

PLAY [Deploy CineBook to EC2] *******
TASK [Install System Packages] ok
TASK [Install Docker Engine] ok
TASK [Create swap file] changed
TASK [Clone CineBook Repo] changed
TASK [Create backend .env] changed
TASK [docker compose up] changed
TASK [Wait for backend] ok

PLAY RECAP ************************
localhost : ok=8 changed=4 failed=0
```

3. Docker Containers

```
docker_ps

$ docker ps --format "table
{{.Names}}\t{{.Status}}"

NAMES               STATUS
cinebook-frontend   Up 12 minutes
(healthy)
cinebook-backend    Up 12 minutes
(healthy)
cinebook-mongo      Up 12 minutes
```

4. API Health Verification

```
verification_script

🚀 Running health check...
Attempt 1/12 - checking api route...
HTTP Code Returned: 200 OK
Response Body Payload:
{
  "status": "success",
  "data": { "moviesCount": 24 }
}
✅ Backend deployment verification
successful!
```

5. Production Endpoint

```
curl_output

$ curl -I http://32.236.37.142

HTTP/1.1 200 OK
Server: nginx/1.25.3
Content-Type: text/html
Cache-Control: no-cache, no-store
Connection: keep-alive

CineBook App Loaded successfully.
```

12 KEY ADVANTAGES

- ✓ **Collaborative Integration**: Webhook integration links branch commits to automated tests, eliminating broken code merges.
- ✓ **Hermetic Environments**: Docker containerization ensures identical package runtimes across dev, staging, and AWS production.
- ✓ **Rapid Configuration**: Ansible handles server configurations dynamically, letting engineers deploy setups in a single command.
- ✓ **Zero Configuration Leaks**: Ansible templates and Jenkins secrets manage security keys, ensuring private keys are never committed.

13 FUTURE ENHANCEMENTS

- **GitOps CD with ArgoCD**: Transition from push-based Jenkins scripts to pull-based declarative synchronization via Git.
- **Canary & Blue-Green Releases**: Integrate traffic splitting with Istio service mesh to safely test new releases with a subset of users.
- **Automated Chaos Engineering**: Inject failures (e.g., using Chaos Mesh) in staging to test auto-healing and recovery metrics.
- **Zero-Trust Secrets Orchestration**: Implement dynamic database credential rotation via HashiCorp Vault.

14 CONCLUSION

The implemented DevOps workflow transitions the **CineBook** application from a fragile manual release process to a modern, robust, and automated continuous deployment pipeline.

By leveraging **Jenkins** for build scheduling, **Ansible** for environment configuration, and **Docker Compose** for microservices orchestration, the system secures reliable releases on **AWS EC2**. The addition of continuous health checks guarantees service integrity, enhancing project collaboration efficiency, security, and cloud scalability.